Sesión 18: Riverpod: Manejo de Estado Complejo (Asincronía con FutureProvider, StreamProvider y Mutación de Estado Inmutable)

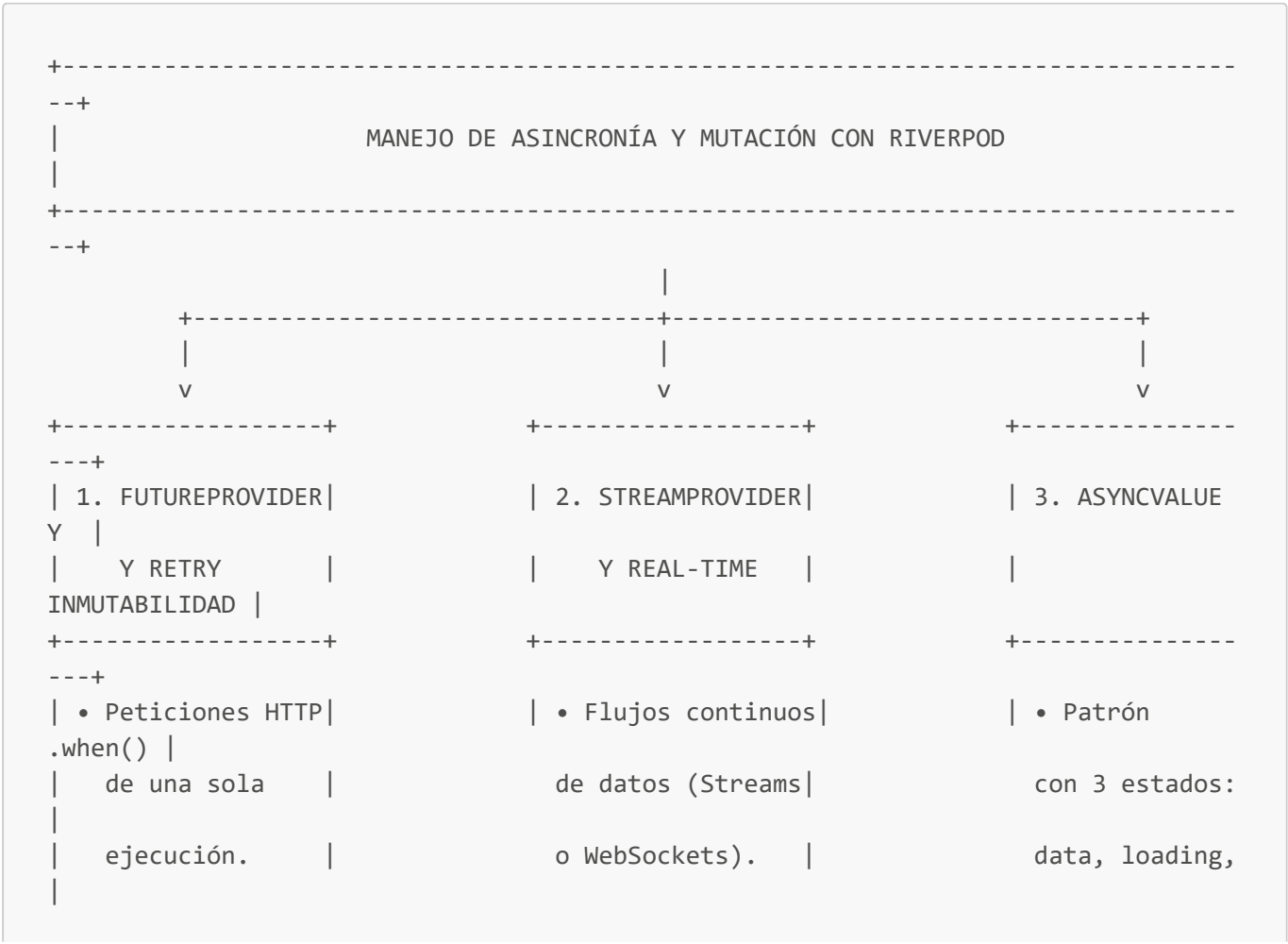
Introducción

En la sesión anterior revisamos los fundamentos de Riverpod y la gestión de datos síncronos mediante `Provider`, `StateProvider` y `NotifierProvider`. Sin embargo, en aplicaciones reales de producción la mayor parte del estado proviene de fuentes de datos asíncronas, tales como peticiones HTTP a APIs REST, consultas a bases de datos SQLite/Isar o flujos de eventos en tiempo real mediante WebSockets y sensores.

En esta sesión exploraremos el manejo de asincronía avanzada en Riverpod mediante `FutureProvider` y `StreamProvider`. Analizaremos la estructura defensiva `AsyncValue`, el patrón de exhaustividad con el método `.when()`, así como los principios para realizar mutaciones de estado inmutable seguro en escenarios complejos.

Conceptos y Definiciones

El manejo de datos asíncronos en Riverpod envuelve las respuestas en el tipo de dato seguro `AsyncValue<T>`, eliminando los errores comunes de puntero nulo (*null pointer exceptions*) durante la carga o el fallo de peticiones.



```
| • Recarga con    |            | • Emisión de     |              error.
|
|   ref.invalidate()|             eventos reactivos|            | • Mutaciones
vía |
|   o ref.refresh().|             en tiempo real.  |              copyWith().
|
+------------------+            +------------------+            +---------------
---+
```

1. Providers Asíncronos: `FutureProvider` y `StreamProvider`

- **`FutureProvider`:**
- Diseñado para peticiones asíncronas puntuales (`Future<T>`), como consumir un endpoint REST o leer un archivo local.
- Maneja el almacenamiento en caché automático de la respuesta y permite invalidar el resultado para forzar una re-petición mediante `ref.invalidate(miFutureProvider)`.
- **`StreamProvider`:**
- Especializado en la escucha de flujos continuos de datos (`Stream<T>`), tales como cambios en tiempo real en bases de datos o lecturas periódicas.
- Transforma cada emisión del *stream* en un estado seguro consumible de manera reactiva por la UI.

---

2. El Envoltorio Seguro `AsyncValue<T>`

`FutureProvider` y `StreamProvider` no devuelven directamente el tipo `T`, sino un tipo envoltorio `AsyncValue<T>`. Esta estructura implementa una unión discriminada que puede encontrarse en uno de tres estados mutuamente excluyentes:

1. **`AsyncData<T>`:** La operación asíncrona finalizó con éxito y contiene los datos.
2. **`AsyncLoading`:** La petición o escucha está en ejecución.
3. **`AsyncError`:** Ocurrió una excepción durante la ejecución asíncrona.

El método `.when()` obliga al desarrollador a controlar explícitamente los tres estados, garantizando que no existan inconsistencias visuales en la interfaz de usuario:

```
asyncValue.when(
  data: (datos) => WidgetConDatos(datos),
  loading: () => const CircularProgressIndicator(),
  error: (err, stack) => WidgetDeError(err),
);
```

---

3. Mutación de Estado Inmutable Estructurado

Al trabajar con colecciones o modelos de datos complejos en la arquitectura reactiva, los objetos nunca deben modificarse de forma directa (mutación por referencia). Se debe aplicar el principio de **inmutabilidad**:

- **Copias Modificadas (`copyWith`):** Para actualizar una propiedad de una entidad, se crea una nueva instancia del objeto conservando las propiedades no modificadas.
- **Operadores Spread (`...`):** En colecciones o listas, se generan nuevas colecciones pasando elementos mediante destructuración, garantizando que el motor de Flutter detecte los cambios de referencia y re-renderice únicamente los componentes suscriptores.

Ejemplo Completo con Código Fuente: Consumo Asíncrono de Eventos y Monitor en Tiempo Real

El siguiente ejemplo simula el consumo de una API REST de publicaciones académicas con `FutureProvider` y una transmisión en tiempo real de métricas del servidor mediante `StreamProvider`, demostrando el uso de `AsyncValue.when()` y la invalidación de caché.

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

void main() {
  runApp(
    const ProviderScope(
      child: DemoRiverpodAsincroniaApp(),
    ),
  );
}

/// Modelo de Datos para Publicaciones Científicas / Académicas
@immutable
class Publicacion {
  final String id;
  final String titulo;
  final String autor;

  const Publicacion({
    required this.id,
    required this.titulo,
    required this.autor,
  });
}

// ============================================================================
// DEFINICIÓN DE PROVEEDORES ASÍNCRONOS (FUTURE & STREAM)
// ============================================================================

/// 1. FUTUREPROVIDER: Petición HTTP simulada para obtener publicaciones
final publicacionesProvider = FutureProvider<List<Publicacion>>((ref) async {
```

```
  // Simulación de latencia de red
  await Future.delayed(const Duration(seconds: 2));

  // Simulación de error aleatorio opcional (para probar estado AsyncError)
  // throw Exception('Error de conexión con el servidor ESCOM.');

  return const [
    Publicacion(
      id: 'PUB-101',
      titulo: 'Arquitecturas Limpias en el Desarrollo Móvil',
      autor: 'M. en C. Ozzy',
    ),
    Publicacion(
      id: 'PUB-102',
      titulo: 'Optimización de Renderizado en Flutter',
      autor: 'Dra. Elena Rostova',
    ),
    Publicacion(
      id: 'PUB-103',
      titulo: 'Seguridad en API REST con OAuth 2.0',
      autor: 'Ing. Carlos Mendoza',
    ),
  ];
});

/// 2. STREAMPROVIDER: Flujo de eventos en tiempo real (Telemetría de Servidor)
final usoCpuStreamProvider = StreamProvider<int>((ref) async* {
  int usoCpu = 45;
  while (true) {
    await Future.delayed(const Duration(seconds: 1));
    // Generación de métrica simulada entre 30% y 95%
    usoCpu = (usoCpu + (DateTime.now().second % 7) - 3).clamp(30, 95);
    yield usoCpu;
  }
});

// ============================================================================
// INTERFAZ DE USUARIO
// ============================================================================

class DemoRiverpodAsincroniaApp extends StatelessWidget {
  const DemoRiverpodAsincroniaApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Asincronía con Riverpod',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
```

```
      home: const PantallaMonitorAsincrono(),
    );
  }
}

class PantallaMonitorAsincrono extends ConsumerWidget {
  const PantallaMonitorAsincrono({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // Lectura del FutureProvider que retorna AsyncValue<List<Publicacion>>
    final estadoPublicaciones = ref.watch(publicacionesProvider);
    // Lectura del StreamProvider que retorna AsyncValue<int>
    final estadoUsoCpu = ref.watch(usoCpuStreamProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Panel de Telemetría y Contenido'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
        actions: [
          IconButton(
            icon: const Icon(Icons.refresh),
            tooltip: 'Invalidar y Recargar Publicaciones',
            onPressed: () {
              // Forzar recarga invalidando la caché del FutureProvider
              ref.invalidate(publicacionesProvider);
            },
          ),
        ],
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // TARJETA DE MONITOR EN TIEMPO REAL (StreamProvider)
            Card(
              color: Theme.of(context).colorScheme.primaryContainer,
              child: Padding(
                padding: const EdgeInsets.all(16.0),
                child: Row(
                  children: [
                    const Icon(Icons.speed, size: 36),
                    const SizedBox(width: 16),
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          const Text(
                            'Telemetría del Servidor (Stream)',
                            style: TextStyle(fontWeight: FontWeight.bold),
                          ),
                          // Manejo exhaustivo de AsyncValue con .when()
```

```
                    estadoUsoCpu.when(
                      data: (porcentaje) => Text(
                        'Carga de CPU: $porcentaje%',
                        style: const TextStyle(fontSize: 16),
                      ),
                      loading: () => const Text('Iniciando sensor...'),
                      error: (err, stack) => Text('Error: $err'),
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
      const SizedBox(height: 20),

      const Text(
        'Publicaciones Científicas (FutureProvider)',
        style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
      ),
      const SizedBox(height: 10),

      // CONSUMO EXHAUSTIVO DEL FUTUREPROVIDER
      Expanded(
        child: estadoPublicaciones.when(
          // Estado 1: Éxito con Datos
          data: (publicaciones) {
            return ListView.builder(
              itemCount: publicaciones.length,
              itemBuilder: (context, index) {
                final pub = publicaciones[index];
                return Card(
                  margin: const EdgeInsets.only(bottom: 8),
                  child: ListTile(
                    leading: const Icon(Icons.article),
                    title: Text(pub.titulo),
                    subtitle: Text(pub.autor),
                    trailing: Text(pub.id),
                  ),
                );
              },
            );
          },
          // Estado 2: En Carga
          loading: () => const Center(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                CircularProgressIndicator(),
                SizedBox(height: 12),
                Text('Consultando repositorio remoto...'),
              ],
            ),
```

```
                ),
                // Estado 3: Error
                error: (error, stackTrace) => Center(
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      const Icon(Icons.error_outline, color: Colors.red, size:
48),
                      const SizedBox(height: 8),
                      Text('Error: $error'),
                      const SizedBox(height: 12),
                      ElevatedButton(
                        onPressed: () => ref.invalidate(publicacionesProvider),
                        child: const Text('Reintentar Petición'),
                      ),
                    ],
                  ),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Prácticas Guiadas / Laboratorio

Práctica 1: Implementación de Búsqueda con Parámetros Usando `FutureProvider.family`

1. Investiga el uso del modificador `.family` en Riverpod, el cual permite pasar argumentos a un proveedor.
2. Declara un `FutureProvider.family<List<Publicacion>, String>((ref, query) => ...)` para filtrar las publicaciones según el texto de búsqueda ingresado por el usuario.
3. Conecta el proveedor con un `TextField` en la interfaz y utiliza `.when()` para mostrar la lista filtrada asíncronamente.

Práctica 2: Simulación de Chat o Registro de Logs en Tiempo Real con `StreamProvider`

1. Crea un controlador de transmisión mediante `StreamController<String>`.
2. Asocia la salida de la transmisión a un `StreamProvider<String>`.
3. Añade un botón en la pantalla que agregue mensajes al `StreamController` y verifica cómo la interfaz de usuario los renderiza de forma reactiva y limpia mediante `AsyncValue`.

Resumen y Puntos Clave

- **Manejo Seguro de Asincronía:** `FutureProvider` y `StreamProvider` abstraen la complejidad de promesas y transmisiones de eventos sin requerir `FutureBuilder` o `StreamBuilder` nativos.
- **`AsyncValue<T>`:** Envoltorio que fuerza la gestión de los estados `data`, `loading` y `error`.
- **Garantía de Exhaustividad:** El método `.when()` evita la Omisión Accidental del renderizado de estados de carga o error.
- **Invalidez de Caché:** `ref.invalidate(provider)` permite re-disparar fácilmente peticiones asíncronas para refrescar interfaces de usuario.

Bibliografía y Referencias

1. **Rousselet, R.** (2024). *Riverpod AsyncValue & FutureProvider Documentation*. Recuperado de: https://riverpod.dev/docs/providers/future_provider
2. **Google LLC.** (2024). *Asynchronous Programming in Dart: Futures and Streams*. Recuperado de: https://dart.dev/codelabs/async-await
3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.